



NewsGraph

Recomendação de notícias financeiras baseada em grafos

Estruturas de Dados 2 • 2026.1 | Temática B — Sistema de recomendação de textos

Apresentação 30/06/2026 github.com/anunesv/Trabalho-EDA2

EQUIPE

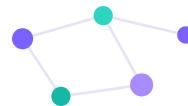
Isabella Lacerda

Paulo Ferreira de Lima Filho

Rodrigo Henrique Donato de Souza

Yasmin Sousa Abdon

Ana Luísa Vieira Nunes



Recomendar a notícia certa que o usuário ainda não leu

Dado um acervo de notícias financeiras e o histórico de interações de vários usuários, recomendar a cada um as notícias mais relevantes que ele ainda não viu — usando exclusivamente estruturas e algoritmos de grafos implementados do zero.

A HIPÓTESE CENTRAL

“Se duas notícias são lidas pelas mesmas pessoas, elas são parecidas. Logo, se você gostou de uma, deve gostar da outra.”

Área de aplicação

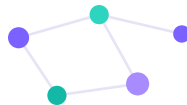
Mercado financeiro — notícias de economia, juros, câmbio, bolsa.

Entrada (texto)

Notícias coletadas por RSS de portais reais; interações de usuários.

Saída

Um ranking Top-N personalizado das notícias não lidas mais relevantes.



Notícias reais, usuários e interações coerentes

● Notícias — reais (RSS)

Coleta de 4 portais por feedparser → filtro de PLN (spaCy) classifica o escopo financeiro → restam as notícias “em escopo”. A coleta roda a cada 3h no GitHub Actions.

● Usuários e interações — via LLM

Fictícios, gerados por Gemini 2.5 Flash com perfis coerentes (renda fixa, trader de ações, cripto/câmbio). Perfis parecidos leem notícias parecidas — é isso que cria a coocorrência de leitores.

47

usuários

136

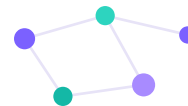
notícias em escopo

~800

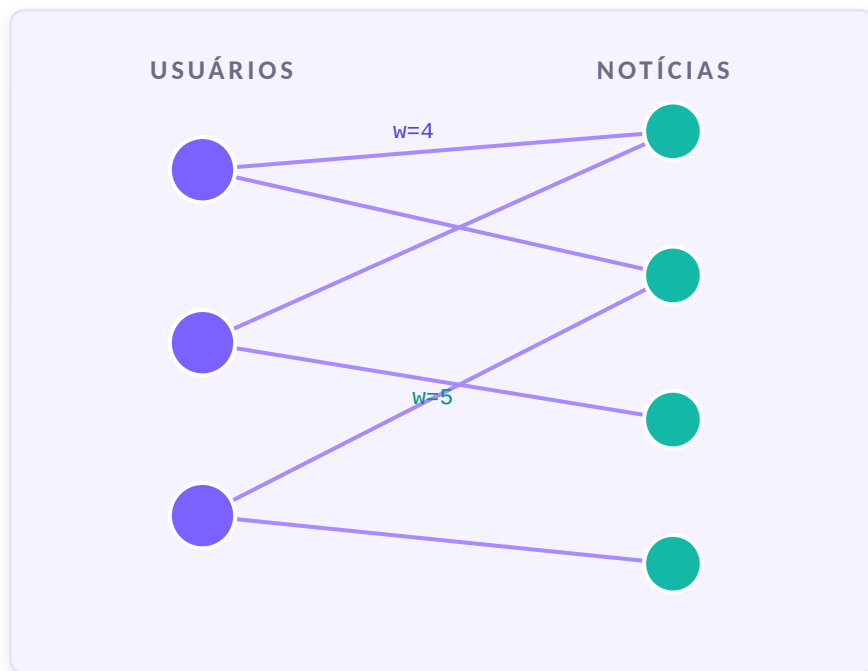
interações

Peso da ação = intensidade do interesse

clique (ler)	+1
like	+4
compartilhar	+5
dislike	-3



Grafo bipartido usuário ↔ notícia



● Vértices

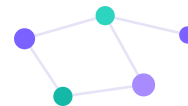
Duas tabelas: usuários e notícias. Notícias guardam título, fonte, link (chave de deduplicação) e os campos do filtro de PLN.

● Arestas

Uma tabela de interações liga usuário → notícia. Cada interação tem um tipo de ação e um peso.

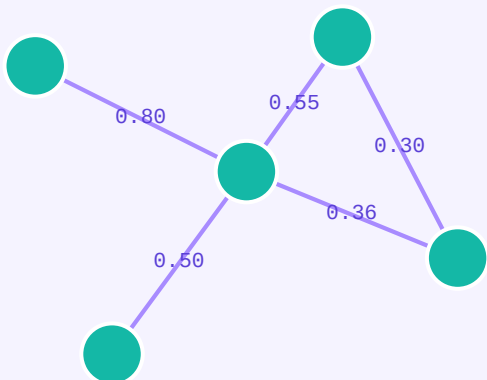
● Pesos

O peso (1 a 5; dislike = -3) codifica a intensidade do interesse e alimenta a similaridade.



Projeção texto ↔ texto por Jaccard ponderado

GRAFO NOTÍCIA ↔ NOTÍCIA

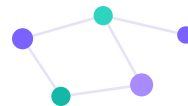


$$J(A,B) = \frac{\sum \min(w_A(u), w_B(u))}{\sum \max(w_A(u), w_B(u))}$$

similaridade de duas notícias a partir de quem as leu

Coocorrência de leitores, não texto. Representa o comportamento real de uma newsletter; é determinística e sem parâmetro a calibrar; e o peso da ação enriquece a medida (uma notícia curtida em comum vale mais que uma só clicada).

1ª técnica de filtragem: pares sem nenhum leitor em comum têm $J = 0$ e são descartados. Com pesos iguais, a fórmula se reduz ao Jaccard clássico $|A \cap B| / |A \cup B|$.



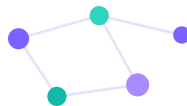
A pipeline: do bipartido ao Top-N



Dois mundos separados pelo banco de dados

OFFLINE (GitHub Actions, sem usuário): coleta RSS → PLN → filtro → LLM gera usuários e interações. É caro e não pode travar a navegação.

ONLINE (Streamlit, com usuário): lê só dados já limpos, monta o grafo em memória, recomenda em segundos e grava as novas interações.



Grafo + duas estruturas auxiliares

1 Grafo

Lista de adjacência (dicionário de dicionários). O grafo é esparso: espaço $O(V+E)$ e acesso a aresta/peso em $O(1)$.

vs. matriz $O(V^2)$, quase toda zerada

2 Union-find

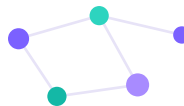
Sustenta o Kruskal: detecção de ciclo em $\sim O(\alpha(n))$, praticamente constante. Compressão de caminho + união por rank.

decide se uma aresta liga 2 grupos

3 Heap Max

Devolve o Top-N sem ordenar a lista inteira, em $O(N \log M)$. É a estrutura adicional exigida além do grafo.

prioridade = (score, -saltos)



Tudo implementado do zero — sem biblioteca de grafos



Jaccard ponderado

similaridade por coocorrência de leitores



Projeção

bipartido $\rightarrow$ grafo notícia $\leftrightarrow$ notícia



Kruskal (máximo)

floresta geradora das ligações mais fortes



Union-find

detecção de ciclo no Kruskal



DFS + gargalo

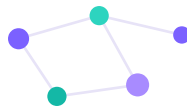
travessia e score do caminho



Heap Max

seleção do Top-N

Bibliotecas só para coleta, PLN, banco e interface. Grafo, Jaccard, Kruskal, union-find, DFS e Heap Max são 100% do grupo — exigência do trabalho (biblioteca pronta de grafos = -5,0).



Stack e arquitetura em camadas

Tecnologias

Python 3.11	todo o projeto
PostgreSQL + psycopg2	persistência (3 tabelas)
feedparser	leitura dos feeds RSS
spaCy (pt_core_news_sm)	PLN: lematização, NER, escopo
Gemini 2.5 Flash	gera dados fictícios (só isso)
Streamlit	login, feed e interações
GitHub Actions	roda a coleta a cada 3h

Camadas e a regra de dependência

app/

interface Streamlit (login, feed, leitura)

recomendacao/

motor.py — orquestra o núcleo para um usuário

core/

NÚCLEO DE GRAFOS — Python puro, algoritmos do zero

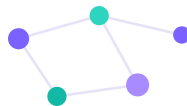
persistencia/

repositorio.py — a ÚNICA porta para o SQL

pipeline/

coleta, PLN, filtro de escopo, geração via LLM

Regra central: o **core/** é puro — recebe dados em memória e devolve resultado.
Ninguém escreve SQL fora de **persistencia/**.



Entra uma interação, sai um ranking — e ele muda

Entrada

As sementes do usuário: as notícias que ele já leu ou curtiu (menos as que recebeu dislike).

Saída

O Top-N de notícias não lidas, cada uma com afinidade (gargalo), número de saltos e recência.

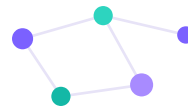
EXEMPLO REAL — usuário 57, ao dar 1 like

Top-5 antes : [33, 90, 42, 20, 57]

Top-5 depois:

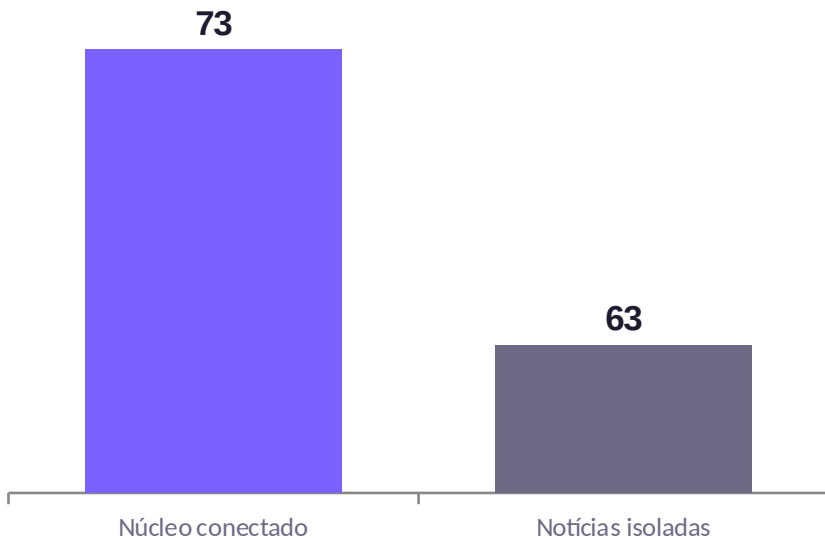
[90, 20, 42, 57, 230]

Ao curtir a notícia 33, ela vira “lida” e **sai do feed**; o ranking se reorganiza e a notícia **230 entra**. O sistema aprende a cada interação.



Um grafo coeso — e vivo

Notícias: núcleo conectado × ilhas



53,7%

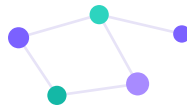
cobertura (73/136)

0,131

Jaccard médio

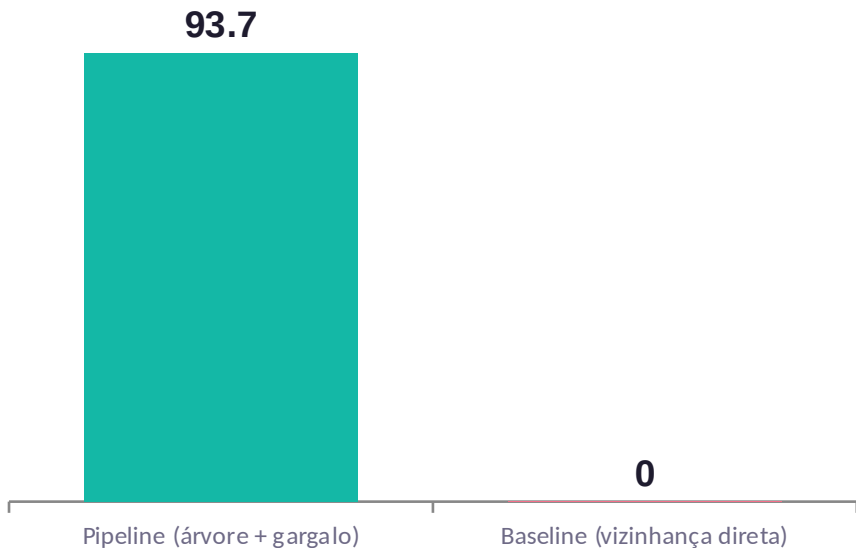
64 componentes = 1 árvore gigante de 73 notícias + 63 singletons. A projeção tinha **2460 arestas**, reduzidas a **72** na árvore.

Sistema vivo. A cobertura caiu porque a coleta traz notícias novas mais rápido do que são lidas — as ilhas se conectam conforme as interações crescem. O Jaccard baixo justifica o Kruskal máximo (2460 → 72).



Por que árvore + gargalo, e não vizinhança direta?

Recomendações a 2+ saltos das sementes (%)



~43%

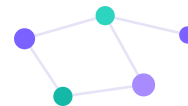
do Top-10 difere

60,4 vs 59,7

alcance — empate

O ganho não é alcançar mais notícias (hoje empatado, efeito do acervo maior). É alcançar as **certas que a vizinhança direta nunca vê**.

93,7% das recomendações vivem a 2+ saltos — notícias que não dividem leitor com nada que o usuário leu. A baseline, por definição, só enxerga 1 salto.



O gargalo manda sobre a distância

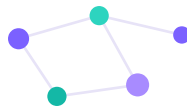
Saltos	Gargalo	Notícia (Top-6 do usuário 57)
1	0,552	'Prévia do PIB' do Banco Central tem alta...
2	0,548	Mercado eleva estimativa de inflação p/ 5,30%...
3	0,548	Acordo de paz EUA-Irã reforça expectativas...
2	0,536	Mercado eleva previsão da Selic...
3	0,519	Senado aprova projeto que blinda agências...
4	0,519	BC amplia acesso a contas em moeda estrangeira...

O caso decisivo

Uma notícia a 3 saltos (gargalo 0,548) aparece **à frente** de uma a 2 saltos (gargalo 0,536): uma conexão indireta mais forte vale mais que uma direta mais fraca.

O nº de saltos só desempata gargalos iguais (os dois 0,548 e os dois 0,519). É a prova de que a árvore + gargalo é mais do que “vizinho mais parecido”.

Ordenado por gargalo (0,552 → 0,519); a coluna de saltos fica fora de ordem (1, 2, 3, 2, 3, 4).



Onde o LLM entra — e onde não entra



No sistema — geração de dados

Gemini 2.5 Flash gera ~40 personas com perfis variados e, para cada uma, 8–20 interações coerentes com o perfil.

- saída estruturada validada por Pydantic
- lotes de 8 usuários por chamada
- perfis parecidos leem notícias parecidas → cria a coocorrência que alimenta o Jaccard



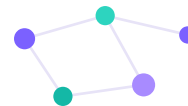
A fronteira — onde NÃO entra

O LLM é usado só na geração de dados fictícios. A recomendação não chama LLM em nenhum ponto.

bipartido · projeção · Jaccard · Kruskal
union-find · DFS + gargalo · Heap Max

→ 100% algoritmo do grupo, sem nenhuma biblioteca de grafos.

No desenvolvimento: assistentes de IA também apoiaram revisão de código e documentação. [ajuste/remova conforme o que a equipe usou]



Quem fez o quê

Frentes mapeadas pela arquitetura do projeto — ajuste a divisão conforme o que cada integrante fez.

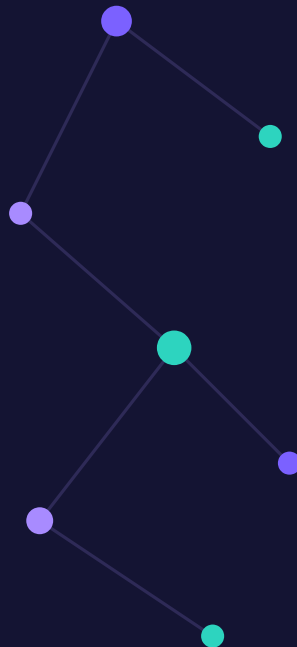
Integrante	Matrícula	Frente principal	Entregas
Isabella Lacerda	232003269	Núcleo de grafos	grafo, Jaccard, projeção, Kruskal + union-find
Paulo Ferreira de Lima Filho	241011976	Busca e ranking	DFS + gargalo, recência, Heap Max (Top-N)
Rodrigo Henrique Donato de Souza	241012374	Pipeline e PLN	coleta RSS, spaCy, filtro de escopo
Yasmin Sousa Abdon	232014271	Dados (LLM) e app	gerador via Gemini, Streamlit (feed)
Ana Luisa Vieira Nunes	232000660	Análise e infra	métricas, ANALISE.md, GitHub Actions

A contribuição final de cada membro será a média das avaliações dos colegas (conforme as regras do trabalho).

CONCLUSÃO

A modelagem em grafos resolve o problema

- 1 **Grafo coeso** — 1 árvore gigante de 73 notícias valida a hipótese de coocorrência de leitores.
- 2 **Kruskal justificado** — Jaccard médio baixo (0,131) → reduzir 2460 arestas a 72 mantendo o essencial.
- 3 **Pipeline ≠ vizinhança direta** — 93,7% das recomendações a 2+ saltos e ~43% do Top-10 diferente — relevância controlada pelo gargalo.
- 4 **Aprende com a interação** — ler, curtir ou descurtir muda o feed na hora; a recência mantém o topo atual.



Não é o app rodando isoladamente — são estes resultados a evidência de que grafos resolvem a recomendação.

Obrigado!